

I. UN PEU D'HISTOIRE...

Au fil du temps et des années, les hommes ont construit des machines de plus en plus élaborées...

- Avant 1900 : les machines mécaniques.
 - ☞ 1642 : machine à calculer de Blaise Pascal (additions et soustractions).
- 1900 à 1939 : les machines électromécaniques
 - ☞ 1935 : IBM 601 (1 multiplication/seconde)
- 1940 à 1955 : les machines à tubes
 - ☞ 1946 : ENIAC (Electronic Numérical Integrator and Computer) : 19000 tubes, 30 tonnes, 72 m², 300 multiplications à la seconde.
- 1947 à 1965 : les machines à transistors
 - ☞ 1947 : invention du transistor
 - ☞ 1954 : TRADIC de Bell (US Air Force)
 - ☞ 1958 : CDC1604, premier ordinateur commercial à base de transistors.
 - ☞ 1958 : démonstration du premier circuit intégré (Jack Kilby, Texas Instrument)
- 1965 à 1977 : les machines à circuits intégrés (CI)
 - ☞ 1971 : premier microprocesseur, le 4004 d'Intel (4 bits, 108 kHz)
- 1977 à nos jours : les micro-ordinateurs
 - ☞ 1977 : Apple II (Apple), PET 2001 (Commodore Business Machines Inc), TRS 80 (Division Radio Shack de Tandy), DAI (Indata)
 - ☞ 1978 : Atari 400 et Atari 800
 - ☞ 1979 : Space invader de Taito
 - ☞ 1980 : Atom (Acorn qui deviendra ARM), ZX 80 (Sinclair Research), Victor (Lambda Systèmes), PC1211 (Sharp)
 - ☞ 1982 : TO7 de Thomson qui a équipé de nombreuses écoles françaises grâce au Plan Informatique pour Tous.

Plus d'informations sur : <http://histoire.info.online.fr/micro.html>

L'histoire des cartes perforées. <https://www.youtube.com/watch?v=f0gONLPtFLQ>

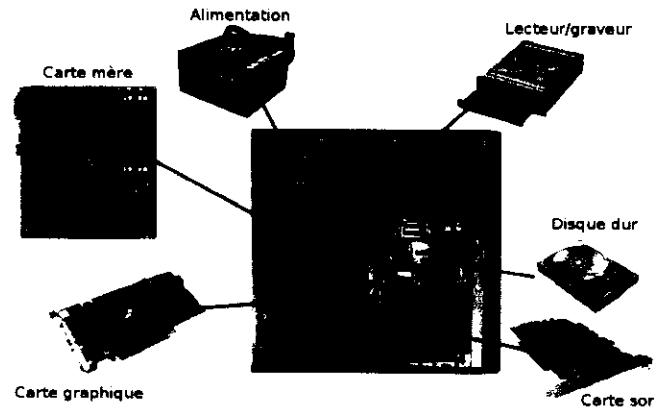
II. ARCHITECTURE MATÉRIELLE

1. Que voit-on quand on ouvre un ordinateur ?

Quand on regarde un ordinateur, on voit :

- **des périphériques externes** : clavier, souris, webcam... (entrées) et écran, imprimante, enceintes... (sorties)
- **l'unité centrale** : l'alimentation, le disque dur, les lecteurs/graveurs, la carte son, la carte graphique, des ventilateurs, des ports externes et la carte mère.

La carte mère, élément central de l'architecture de l'ordinateur accueille : le processeur (dans un emplacement appelé socket), la mémoire vive (sous forme de barrettes enfichées sur des slots dédiés à la RAM)), les connecteurs de disques durs, de lecteurs CD/DVD, la pilaire (permet d'alimenter l'horloge de l'ordinateur, quand il est éteint, et conserver certains paramètres du BIOS), des slots PCI (pour connecter certaines cartes son, carte graphiques, carte réseau...), des Chipsets (composants électroniques permettant de gérer les flux de données numériques entre le processeur, la mémoire et les périphériques), des ports externes (USB, VGA, HDMI...)

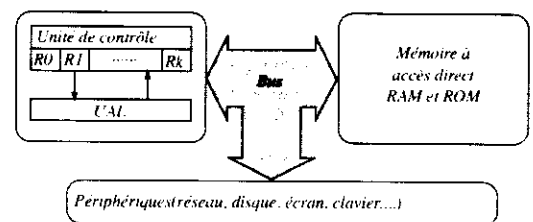


2. L'architecture de Von Neumann

Les ordinateurs d'aujourd'hui sont encore conçus selon l'architecture de Von Neumann (travaux faits durant la seconde guerre mondiale).

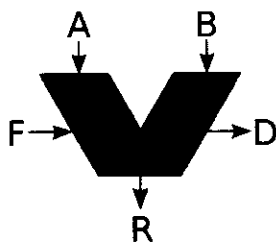
Une machine suivant le modèle de Von Neumann est constituée de :

- ☞ une **unité centrale** composée d'une unité de calcul (UAL : Unité Arithmétique et Logique) et d'une unité de contrôle ou de commande. le tout est rythmé par une horloge interne qui détermine la fréquence du processeur ;
- ☞ une **mémoire centrale** composée d'un ensemble de cellules stockant des nombres binaires représentant les programmes et les données ;
- ☞ un **ensemble de périphériques** permettant à la machine d'interagir avec le monde extérieur ;
- ☞ un **canal de communication** entre ces 3 entités, appelé **Bus**, composé de simples fils.



a. L'UAL

C'est le cœur de l'ordinateur. On la représente habituellement par ce schéma :



- A et B sont les opérandes (les entrées)
- R est le résultat
- F est une fonction binaire (l'opération à effectuer)
- D est un drapeau indiquant un résultat secondaire de la fonction (signe, erreur, division par 0...)

Les **opérations** que peut réaliser une UAL de base sont :

- ☞ les additions, soustractions, multiplications et divisions ;
- ☞ les opérations logiques (ET, OU, NON)
- ☞ les opérations de comparaison (<, > et =).

Certaines sont spécialisées (calculs sur les nombres à virgule flottante, cartes graphiques...).

Un même processeur peut contenir plusieurs UAL. Lorsque ces UAL fonctionnent en parallèles, on parle d'architecture **multi-cœur**. Ceci permet d'augmenter la puissance de calcul.

b. La mémoire

C'est un tableau dans lequel sont stockés les données et les programmes.

Chaque cellule de ce tableau possède une adresse X et la valeur de chaque cellule est notée M[X] ou #X ou \$X

Différentes mémoires existent :

- ☞ **la mémoire interne vive (RAM : Random Access Memory)** dite mémoire principale : souvent ce sont des barrettes que l'on peut rajouter ou enlever. Elle est **volatile et rapide**. Elle est accessible en mode **lecture et écriture**.
- ☞ **la mémoire interne morte (ROM : Read Only Memory)** : elle contient des programmes qui se lancent automatiquement au démarrage de l'ordinateur et indispensables au bon fonctionnement de tous les éléments de l'ordinateur. Elle est **permanente et plus lente** que la RAM. Elle est accessible uniquement **en lecture**.
- ☞ **la mémoire externe** : disques durs, clés USB... Elle est **permanente, lente**. (temps d'accès de l'ordre de 50 microsecondes pour un disque dur SSD à quelques millisecondes pour un disque dur HDD). Elle est accessible en **lecture et écriture**.

c. L'unité de commande

C'est elle qui donne les ordres à toutes les autres parties de l'ordinateur et gère l'ordre dans lequel se font les opérations.

Elle mémorise les informations dans des registres, mémoire internes au processeur d'accès très rapide (nanoseconde) et volatiles. Ils permettent de mémoriser des données et instructions au sein même du processeur.

Elle contient deux registres importants qui ne sont habituellement pas accessibles aux programmeurs :

- Le registre d'instruction (RI) contient sous forme codée l'instruction machine en cours d'exécution.
- Le registre compteur ordinal (CO) contient l'adresse de l'emplacement mémoire de la prochaine instruction à exécuter.

L'accumulateur (ACC) permet de stocker les opérandes ainsi que les résultats des opérations traitées par l'UAL ; Il est cloisonné en plusieurs registres souvent notés R1, R2, R3... L'unité de commande décode les instructions d'un programme et les transcrit en une succession d'instructions élémentaires envoyées à l'unité de traitement dont fait partie l'UAL. Son travail est cadencé par une horloge, ce qui permet de coordonner les instructions et d'optimiser le traitement d'une succession d'instructions.

La vitesse de battement de cette horloge s'appelle la fréquence (1 battement = 1 opération).
Un processeur qui a une fréquence de 3 GHz peut effectuer 3 milliards d'opérations de base par seconde.

Le fonctionnement d'un processeur repose sur l'algorithme suivant :

Tant que CO contient l'adresse d'une instruction :
 RI reçoit de la mémoire principale l'instruction dont l'adresse est dans CO
 L'instruction chargée dans RI est exécutée #Entrée en scène de ACC et UAL
 CO reçoit l'adresse suivante.

d. Le bus

Il assure la circulation des données entre les différentes parties d'un ordinateur, notamment la mémoire principale et le CPU.

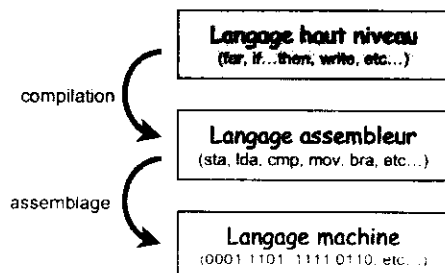
- Le bus d'adresse permet de faire circuler des adresses : par exemple, l'adresse d'une donnée en mémoire.
- Le bus de données permet de faire circuler des données.
- Le bus de contrôle permet de spécifier le type d'action (écriture d'une donnée, lecture d'une donnée en mémoire).

3. Fonctionnement

Un ordinateur ne fait que ce qu'on lui demande de faire. Son langage est un langage binaire (des 0 et des 1). Son fonctionnement repose sur des circuits intégrés, gravés sur du silicium, composés de transistors. Pour communiquer avec l'ordinateur, nous utilisons des langages de programmation, traduits ensuite par des compilateurs ou assembleur.

Ces circuits intégrés sont conçus de façon à permettre la traduction des opérations simples : le ET, le OU logiques, l'inversion d'un bit, l'addition de 2 bits. Ces opérations combinées entre-elles donnent des fonctions ou portes logiques (Le logiciel logisim permet de simuler ces portes logiques).

En résumé :



Un exemple :

Haut niveau
Langage humain : "Si x est plus grand que 10, alors décrémenter x..."
Langage haut niveau (C, PHP, ...): "if(x > 10) x--;"
Langage assembleur : "cmp x, 10 dec: dec x jb dec "
Langage machine : "001110100110110110101011101 0000110001101110110110011001"
Bas niveau

Le **langage machine** est le langage compris par le microprocesseur. Ce langage est difficile à maîtriser puisque chaque instruction est codée par une séquence propre de bits. Afin de faciliter la tâche du programmeur, on a créé différents langages plus ou moins évolués.

Le **langage assembleur** est le langage le plus « proche » du langage machine. Il est composé par des instructions en général assez rudimentaires que l'on appelle des mnémoniques. Ce sont essentiellement des opérations de transfert de données entre les registres et l'extérieur du microprocesseur (mémoire ou périphérique), ou des opérateurs arithmétiques ou logiques. Chaque instruction représente un code machine différent. Chaque microprocesseur peut posséder un assembleur différent.

4. Exemples d'instructions passées à la machine lors de l'écriture d'un programme en Python

Le module `dis` de Python permet d'avoir une idée des instructions passées à la machine lorsque nous écrivons un petit programme en Python :

	1	0 LOAD_CONST	0 (1)
		2 STORE_NAME	0 (x)
		4 LOAD_NAME	0 (x)
1	<code>import dis</code>	6 LOAD_CONST	1 (2)
2	<code>dis.dis("x=1;x=x+2")</code>	8 BINARY_ADD	
		10 STORE_NAME	0 (x)
		12 LOAD_CONST	2 (None)
		14 RETURN_VALUE	

En prenant ces instructions dans l'ordre :

- ☞ 0. Le nombre 1 est copié dans un registre
- ☞ 2. Le contenu du registre est copié dans la mémoire à la case adressée par `x`
- ☞ 4. La valeur de `x` est copié dans un registre
- ☞ 6. Le nombre 2 est copié dans un registre
- ☞ 8. L'addition des 2 est exécutée
- ☞ 10. Le résultat est copié dans la mémoire à la case adressée par `x`
- ☞ 12. La valeur `None` est copiée dans un registre
- ☞ 14. Elle est renvoyée

Dans un langage assembleur, on obtiendrait la suite d'instructions :

```
MOV R0,#1
STR R0, x
LDR R0, x
MOV R1, #2
ADD R0, R0, R1
STR R0, x
```

Un autre exemple :

	2	0 LOAD_CONST	0 (3)	
		2 STORE_NAME	0 (x)	
1	<code>import dis</code>			
2		3	4 LOAD_NAME	0 (x)
3	<code>code= """</code>		6 LOAD_CONST	1 (0)
4	<code>x=3</code>		8 COMPARE_OP	0 (<)
5	<code>if x<0:</code>		10 POP_JUMP_IF_FALSE	20
6	<code> y=-x</code>	4	12 LOAD_NAME	0 (x)
7	<code>else:</code>		14 UNARY_NEGATIVE	
8	<code> y=x</code>		16 STORE_NAME	1 (y)
9	<code>"""</code>		18 JUMP_FORWARD	4 (to 24)
10	<code>dis.dis(code)</code>	6	>> 20 LOAD_NAME	0 (x)
			22 STORE_NAME	1 (y)
		>>	24 LOAD_CONST	2 (None)
			26 RETURN_VALUE	

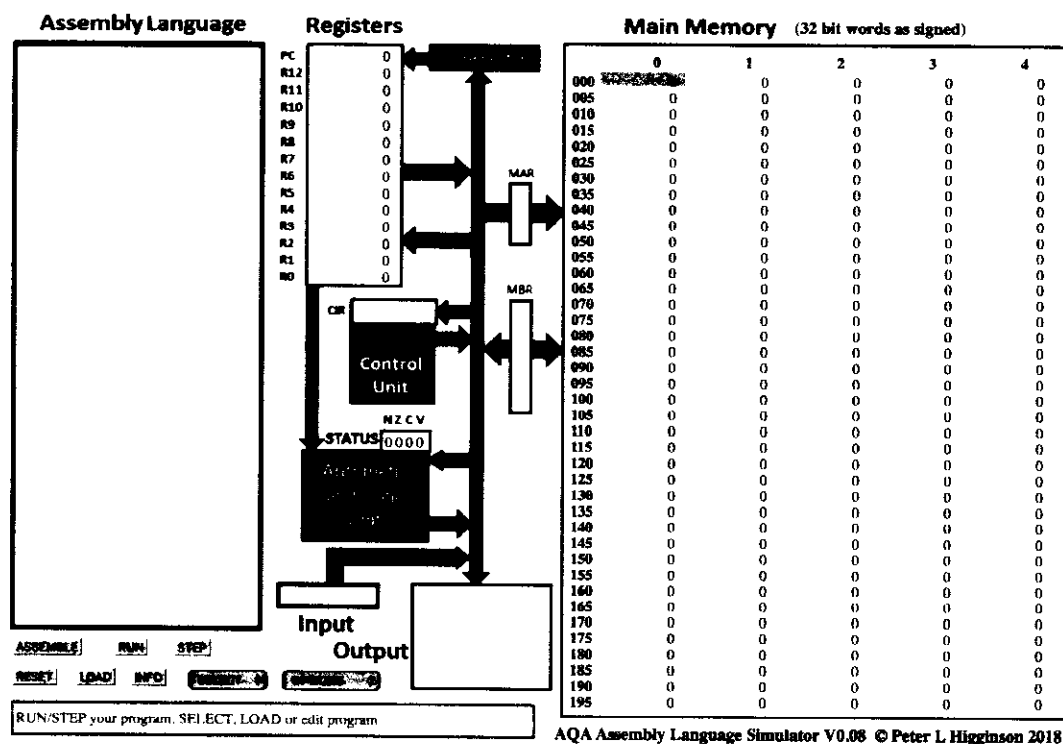
5. Des vidéos à voir

- ☞ Comment fonctionne un ordinateur. <https://www.youtube.com/watch?v=85XUJXHbjBo>
- ☞ Les portes logiques <https://www.youtube.com/watch?v=iTH39L2d7bg>
- ☞ La notion de cache en informatique <https://www.youtube.com/watch?v=bcB0yAlU2m4>

Dans ce TP, nous allons simuler le fonctionnement de la machine de Von Neumann.

I. LE SIMULATEUR

Ouvrir la page <http://www.peterhigginson.co.uk/AQA/>. Vous devriez voir :



- À droite, on trouve la mémoire vive (Main Memory)
- Au centre, on trouve le microprocesseur.
- À gauche, on trouve la zone où nous allons saisir nos programme en assembleur.

LA RAM : par défaut, le contenu des différentes cellules de la mémoire vive (RAM) est en base 10 (entier signé). D'autres options sont possibles à l'aide du bouton **OPTIONS**.

- À FAIRE** : à l'aide du bouton **OPTIONS**, passez à un affichage binaire.

Vous pouvez constater que chaque cellule de la mémoire comporte 32 bits. Chaque cellule de la mémoire possède une adresse (de 000 à 199) et ces adresses sont codées en base 10. Vous pouvez repasser à un affichage en base 10 en choisissant "signed" dans **OPTIONS**.

Le CPU : dans la partie centrale du simulateur, nous trouvons :

- le bloc "registre" ("Registers") qui comporte 13 registres de R0 à R12 et 1 registre PC qui contient l'adresse mémoire de l'instruction en cours d'exécution.
- le bloc "unité de commande" ("Control Unit") qui contient l'instruction machine en cours d'exécution (au format hexadécimal).
- le bloc "unité arithmétique et logique" (Arithmetic and Logic Unit).

II. PROGRAMMER EN ASSEMBLEUR

1. Dans la partie gauche, saisir ces 3 lignes de code en assembleur :

```
MOV R0,#42
STR R0,150
HALT
```

La première ligne place le nombre 42 dans le registre R0 ;
la deuxième stocke le contenu de R0 dans la mémoire 150
et la troisième arrête le processus.

2. Une fois la saisie terminée, cliquez sur le bouton **submit**. Vous devriez voir apparaître des nombres dans les cellules mémoire 000, 001, 002 :

Main Memory (32 bit words as signed)				
	0	1	2	4
000	42	-443612596	-285212672	0
005	0	0	0	0

L'assembleur a fait son travail : il a converti les 3 lignes de notre programme en instructions machines. La première instruction machine est stockée à l'adresse mémoire 000, la deuxième à l'adresse 001 et la troisième à l'adresse 002.

On peut faire afficher les instructions en binaire (OPTIONS) pour avoir une idée des véritables instructions machines :

Main Memory (32 bit words as binary)		
	0	1
000	00000000 00000000 00000000 00000000	11100101 10001111 00000010 01001100
005	00000000 00000000 00000000 00000000	00000000 00000000 00000000 00000000

On peut associer chaque instruction machine à son code en assembleur.

On peut remarquer que la cellule 000, l'octet le plus à droite est bien égal à 42.

3. Pour exécuter notre programme, il suffit maintenant de cliquer sur le bouton "RUN". Vous allez voir le CPU "travailler" en direct grâce à de petites animations. Si cela va trop vite (ou trop doucement), vous pouvez régler la vitesse de simulation à l'aide des boutons "<" et ">". Un appui sur le bouton "STOP" met en pause la simulation, si vous appuyez une deuxième fois sur ce même bouton "STOP", la simulation reprend là où elle s'était arrêtée.

Une fois la simulation terminée, vous pouvez constater que la cellule mémoire d'adresse 150, contient bien le nombre 42 (en base 10). Vous pouvez aussi constater que le registre R0 a bien stocké le nombre 42, en hexadécimal..

ATTENTION : pour relancer la simulation, il est nécessaire d'appuyer sur le bouton "RESET" afin de remettre les registres R0 à R12 à 0, ainsi que le registre PC (il faut que l'unité de commande pointe de nouveau sur l'instruction située à l'adresse mémoire 000). La mémoire n'est pas modifiée par un appui sur le bouton "RESET", pour remettre la mémoire à 0, il faut cliquer sur le bouton "OPTIONS" et choisir "clr memory". Si vous remettez votre mémoire à 0, il faudra cliquer sur le bouton "ASSEMBLE" avant de pouvoir exécuter de nouveau votre programme.

4. Dans le programme précédent, rajouter 2 lignes pour qu'à la fin de l'exécution on trouve le nombre 54 à l'adresse mémoire 50. On utilisera le registre R1 à la place du registre R0. Testez vos modifications en exécutant la simulation.

Ajouter ensuite la ligne `ADD R2,R0,R1`. Commentez. *→ le registre R2 contient l'addition des valeurs initiales dans les registres R0 et R1. R2 vaut 54+42=96.*

5. Quelques compléments :

- `LDR Rd, <adresse mémoire>` Charge la valeur enregistrée dans l'<adresse mémoire > dans le registre d
- `STR Rd, <adresse mémoire>` Enregistre la valeur du registre d dans la mémoire spécifiée par <adresse mémoire > -

- ADD Rd, Rn, <operand2> ajoute la valeur spécifiée par <operand2> à la valeur du registre d et enregistre le résultat dans le registre d
- SUB Rd, Rn, <operand2> Soustrait la valeur de <operand2> à la valeur du registre n et enregistre le résultat dans le registre d
- MOV Rd, <operand2> Copie la valeur <operand2> dans le registre d
- CMP Rn, <operand2> Compare la valeur de registre n avec la valeur de <operand2>.
- B <label> Branchement inconditionnel jusqu'à la position <label> dans le programme.
- B <condition> <label> Branchement conditionnel vers la position <label> dans le programme si la dernière comparaison remplit le critère spécifié par <condition>. Les valeurs possibles sont : EQ :égal à ; NE : n'est pas égal à ; GT :Plus grand que ; LT : Moins grand que.
- AND Rd, Rn, <operand2> Effectue l'opération bit à bit logique AND (ET) entre la valeur du registre n et la valeur <operand2> et enregistre le résultat dans le registre d.
- ORR Rd, Rn, <operand2> Effectue l'opération bit à bit logique OR (OU) entre la valeur du registre n et la valeur <operand2> et enregistre le résultat dans le registre d.
- EOR Rd, Rn, <operand2> Effectue l'opération bit à bit logique XOR(OU exclusif) entre la valeur du registre n et la valeur <operand2> et enregistre le résultat dans le registre d.
- MVN Rd, <operand2> Effectue l'opération bit à bit logique NOT (NON) sur la valeur <operand2> et enregistre le résultat dans le registre d.
- HALT Arrête l'exécution du programme.
- <operand2> peut être #nnn (c'est à dire un nombre, exemple #42) ou bien Rm (c'est à dire le registre m, par exemple R1 est le registre numéro 1)
- la pseudo instruction DAT vous permet de mettre un nombre dans la mémoire en utilisant l'assembleur. Un label peut aussi être pris comme donnée.
- INP Rd,2 lit un nombre dans le registre d.
- OUT Rd,4 retourne en sortie le nombre du registre d

a. Que font les 2 codes ci-contre ? Vérifier.

*teste si R2 et R1
sont égaux
si non la boucle
du test*

```
MOV R0,#13
MOV R1,#11
AND R2,R0,R1
OUT R2,4
HALT
```

```
MOV R0,#101
MOV R1,#11
ORR R2,R0,R1
OUT R2,4
HALT
```

b. Que fait le code ci-contre ? Comment le modifier pour faire l'inverse du programme ?

```
INP R0,2
INP R1,2
CMP R1,R0
BGT HIGHER
OUT R0,4
B DONE
HIGHER :
OUT R1,4
DONE :
HALT
```

c. Le code ci-contre permet de calculer 2^5 à l'aide d'une boucle **while**. Le comprendre et le modifier pour qu'il affiche 2^{10}

```
MOV R0,#2
MOV R1,#1
B maboucle
maboucle :
ADD R0,R0,R0
ADD R1,R1,#1
CMP R1,#5
BNE maboucle
STR R0,100
HALT
```

d. Écrire en assembleur un programme qui calcule la somme des n premiers entiers $1+2+3+4+5+...+n$